

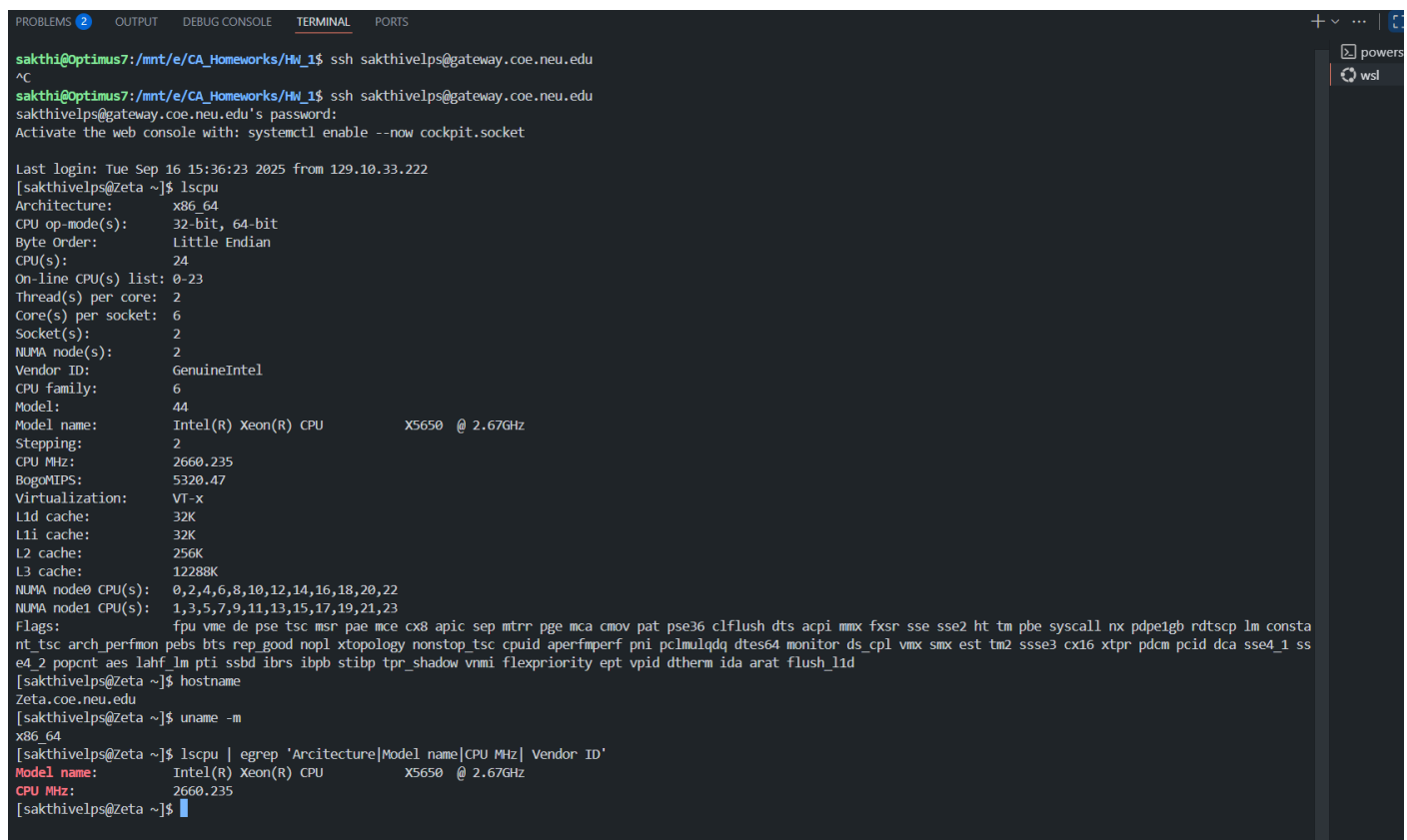
EECE 7352 Computer Architecture

Homework 1

Part A:

In this part of the assignment, Dhrystone (dry.c) a Synthetic benchmark program is used for benchmarking two different architectures X86_64 and ARM_V8 (64 bit) with different optimization levels (**O0 to O3**) and using 2 different compilers (**GCC and Clang**) and evaluating their performances.

System information:



```
sakthi@Optimus7:/mnt/e/CA_Homeworks/Hw_1$ ssh sakthivelp@gateway.coe.neu.edu
AC
sakthi@Optimus7:/mnt/e/CA_Homeworks/Hw_1$ ssh sakthivelp@gateway.coe.neu.edu
sakthivelp@gateway.coe.neu.edu's password:
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Tue Sep 16 15:36:23 2025 from 129.10.33.222
[sakthivelp@zeta ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                24
On-line CPU(s) list:   0-23
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):             2
NUMA node(s):          2
Vendor ID:             GenuineIntel
CPU family:            6
Model:                44
Model name:            Intel(R) Xeon(R) CPU           X5650  @ 2.67GHz
Stepping:              2
CPU MHz:               2660.235
BogoMIPS:              5320.47
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              12288K
NUMA node0 CPU(s):     0,2,4,6,8,10,12,14,16,18,20,22
NUMA node1 CPU(s):     1,3,5,7,9,11,13,15,17,19,21,23
Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ht tm pbe syscall nx pdpe1gb rdtscp lm const
nt_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 cx16 xtpr pdcm pcid dca sse4_1 ss
e4_2 popcnt aes lahf_lm pti ssbd ibrs ibpb stibp tpr_shadow vmmi flexpriority ept vpid dtherm ida arat flush_l1d
[sakthivelp@zeta ~]$ hostname
zeta.coe.neu.edu
[sakthivelp@zeta ~]$ uname -m
x86_64
[sakthivelp@zeta ~]$ lscpu | egrep 'Architecture|Model name|CPU MHz| Vendor ID'
Model name:            Intel(R) Xeon(R) CPU           X5650  @ 2.67GHz
CPU MHz:               2660.235
[sakthivelp@zeta ~]$
```

Given that we have the source code file (**dry.c**), next step is to compile and run the program. Which I have done my using the commands mentioned in the given homework document.

gcc -c -Wno-old-style-definition dry.c -o dry1.o

gcc -DPASS2 -Wno-old-style-definition dry.c dry1.o -o dry22a

Building Dry.c

```
[sakthivelps@Iota PartA]$ gcc -c -O0 -Wno-old-style-definition dry.c -o dry1_00.o
[sakthivelps@Iota PartA]$ gcc -DPASS2 -O0 -Wno-old-style-definition dry.c dry1_00.o -o dry_00
[sakthivelps@Iota PartA]$ ./dry_00
```

```
Dhrystone Benchmark, Version C, Version 2.2a
Program compiled without 'register' attribute
using times(), HZ=100
```

```
Trying 50000 runs: too few
Trying 500000 runs: too few
Trying 5000000 runs: too few
Final values of the variables used in the benchmark:
```

```
Int_Glob:      5
  should be:   5
Bool_Glob:     1
  should be:   1
Ch_1_Glob:     A
  should be:   A
Ch_2_Glob:     B
  should be:   B
Arr_1_Glob[8]: 7
  should be:   7
Arr_2_Glob[8][7]: 55550010
  should be:   Number_Of_Runs + 10
Ptr_Glob->
  Ptr_Comp:    32387744
  should be:   (implementation-dependent)
Discr:        0
  should be:   0
Enum_Comp:    2
  should be:   2
Int_Comp:     17
```

Compiled the benchmark with both **GCC** and **Clang** in 4 different levels of optimization (O0, O1, O2, O3). Ran 10 iterations for each level of optimization and have reported the best run below. I didn't use other optimization levels for this assignment as I was able to get enough information for comparison purposes within the suggested set of optimization levels.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● bash-4.4$ python3 PartA/X86_64_GCC_and_Clang/find_best_runs.py PartA/X86_64_GCC_and_Clang/runs_GCC --csv PartA/X86_64_GCC_and_Clang/runs_GCC/runs_summary_gcc.csv

Best iterations per log:
File                                Iter  Dhrystones/sec
-----
run_22a.log                          5      9596929
run_22a_00.log                       3      9578544
run_22a_01.log                       7     25826446
run_22a_02.log                       8     31525850
run_22a_03.log                       2     31328320

Wrote CSV summary to: PartA/X86_64_GCC_and_Clang/runs_GCC/runs_summary_gcc.csv
● bash-4.4$ python3 PartA/X86_64_GCC_and_Clang/find_best_runs.py PartA/X86_64_GCC_and_Clang/runs_Clang --csv PartA/X86_64_GCC_and_Clang/runs_Clang/runs_summary_clang.csv

Best iterations per log:
File                                Iter  Dhrystones/sec
-----
run_22a_00_clang.log                 1     9881423
run_22a_01_clang.log                 1     22222222
run_22a_02_clang.log                 8     33967392
run_22a_03_clang.log                 6     33783784
run_22a_clang.log                    4     9900990

Wrote CSV summary to: PartA/X86_64_GCC_and_Clang/runs_Clang/runs_summary_clang.csv
○ bash-4.4$
```

Used a python script to extract the best runs under different optimization levels.

Using GCC:

Optimization lvl	Iteration number	Dhrystones/sec
O0	7	9578544
O1	2	25826446
O2	8	31525850
O3	2	31328320

For Clang:

Optimization lvl	Iteration number	Dhrystones/sec
O0	1	9881423
O1	6	22222222
O2	8	33967392
O3	6	33738784

Overall Run Observation (both GCC and Clang)

- **O0** - (This is the **slowest of all, no optimization**, reason: every variable is staying in the memory, no instruction reordering and absence of inlining.
- **O1** – The code is almost **2.5x faster than the previous one (O1)** , some basic optimizations are done such as constant folding, register allocation, dead code elimination, simple instruction scheduling.
- **O2** – This gives the **peak performance**, reason: inlining, instruction reordering, loop unrolling, thus making the code size optimal for faster execution.
- **O3** – though O3 is expected to be better than O2, introducing aggressive inlining, vectorization, loop transformations can sometime bloat the code causing more cache misses.

GCC vs Clang

- **Clang** slightly edges out GCC (eg: for O2 , clang – 33.9M vs GCC- 31.5M)
- This slight difference stems from the optimization heuristics of each compiler, Clang tends to generate **Tighter loops and better vectorization**, whereas GCC often focuses on broader H/W compatibility.

Assembly level comparison of different optimizations (in GCC)

00

```
PartA > X86_64_GCC_and_Clang > asm-snippets_x86_64_GCC > asm dry_00.s
978 .LFE6:
981 .type Proc_1, @function
982 Proc_1:
983 .LFB7:
984 .cfi_startproc
985 pushq %rbp #
986 .cfi_def_cfa_offset 16
987 .cfi_offset 6, -16
988 movq %rsp, %rbp #,
989 .cfi_def_cfa_register 6
990 pushq %rbx #
991 subq $40, %rsp #,
992 .cfi_offset 3, -24
993 movq %rdi, -40(%rbp) # Ptr_Val_Par, Ptr_Val_Par
994 # dry.c:753: REG Rec_Pointer Next_Record = Ptr_Val_Par->Ptr
995 movq -40(%rbp), %rax # Ptr_Val_Par, tmp111
996 movq (%rax), %rax # Ptr_Val_Par_16(D)->Ptr_Comp, t
997 movq %rax, -24(%rbp) # tmp112, Next_Record
998 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Gl
999 movq Ptr_Glob(%rip), %rdx # Ptr_Glob, Ptr_Glob.95_
1000 movq -40(%rbp), %rax # Ptr_Val_Par, tmp113
1001 movq (%rax), %rax # Ptr_Val_Par_16(D)->Ptr_Comp, _
1002 movq (%rdx), %rcx # *Ptr_Glob.95_1, tmp114
1003 movq 8(%rdx), %rbx # *Ptr_Glob.95_1,
1004 movq %rcx, (%rax) # tmp114, *_2
1005 movq %rbx, 8(%rax) #, *_2
1006 movq 16(%rdx), %rcx # *Ptr_Glob.95_1, tmp115
1007 movq 24(%rdx), %rbx # *Ptr_Glob.95_1,
1008 movq %rcx, 16(%rax) # tmp115, *_2
1009 movq %rbx, 24(%rax) #, *_2
1010 movq 32(%rdx), %rcx # *Ptr_Glob.95_1, tmp116
1011 movq 40(%rdx), %rbx # *Ptr_Glob.95_1,
1012 movq %rcx, 32(%rax) # tmp116, *_2
1013 movq %rbx, 40(%rax) #, *_2
1014 movq 48(%rdx), %rdx # *Ptr_Glob.95_1, tmp117
1015 movq %rdx, 48(%rax) # tmp117, *_2
1016 # dry.c:759: Ptr_Val_Par->variant.var_1.Int_Comp = 5;
```

01

```
PartA > X86_64_GCC_and_Clang > asm-snippets_x86_64_GCC > asm dry_01.s
60 .LFE25:
63 .type Proc_1, @function
64 Proc_1:
65 .LFB23:
66 .cfi_startproc
67 pushq %rbp #
68 .cfi_def_cfa_offset 16
69 .cfi_offset 6, -16
70 pushq %rbx #
71 .cfi_def_cfa_offset 24
72 .cfi_offset 3, -24
73 subq $8, %rsp #,
74 .cfi_def_cfa_offset 32
75 movq %rdi, %rbp # Ptr_Val_Par, Ptr_Val_Par
76 # dry.c:753: REG Rec_Pointer Next_Record = Ptr_Val_Par->Ptr_Comp;
77 movq (%rdi), %rbx # Ptr_Val_Par_14(D)->Ptr_Comp, Next_Record
78 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Glob);
79 movq Ptr_Glob(%rip), %rax # Ptr_Glob, Ptr_Glob
80 movdqu (%rax), %xmm0 # *Ptr_Glob.95_1, tmp127
81 movups %xmm0, (%rbx) # tmp127, *Next_Record_15
82 movdqu 16(%rax), %xmm1 # *Ptr_Glob.95_1, tmp128
83 movups %xmm1, 16(%rbx) # tmp128, *Next_Record_15
84 movdqu 32(%rax), %xmm2 # *Ptr_Glob.95_1, tmp129
85 movups %xmm2, 32(%rbx) # tmp129, *Next_Record_15
86 movq 48(%rax), %rax # *Ptr_Glob.95_1, *Ptr_Glob.95_1
87 movq %rax, 48(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
88 # dry.c:759: Ptr_Val_Par->variant.var_1.Int_Comp = 5;
89 movl $5, 16(%rdi) #, Ptr_Val_Par_14(D)->variant.var_1.Int_Comp
90 # dry.c:761: = Ptr_Val_Par->variant.var_1.Int_Comp;
91 movl $5, 16(%rbx) #, Next_Record_15->variant.var_1.Int_Comp
92 # dry.c:762: Next_Record->Ptr_Comp = Ptr_Val_Par->Ptr_Comp;
93 movq (%rdi), %rax # Ptr_Val_Par_14(D)->Ptr_Comp, _2
94 # dry.c:762: Next_Record->Ptr_Comp = Ptr_Val_Par->Ptr_Comp;
95 movq %rax, (%rbx) # _2, Next_Record_15->Ptr_Comp
96 # dry.c:763: Proc_3 (&Next_Record->Ptr_Comp);
97 movq %rbx, %rdi # Next_Record,
98 call Proc_3 #
```

02

```
PartA > X86_64_GCC_and_Clang > asm-snippets_x86_64_GCC > asm dry_02.s
978 .LFE6:
981 .type Proc_1, @function
982 Proc_1:
983 .LFB7:
984 .cfi_startproc
985 pushq %rbp #
986 .cfi_def_cfa_offset 16
987 .cfi_offset 6, -16
988 movq %rdi, %rbp # Ptr_Val_Par, Ptr_Val_Par
989 pushq %rbx #
990 .cfi_def_cfa_offset 24
991 .cfi_offset 3, -24
992 subq $8, %rsp #,
993 .cfi_def_cfa_offset 32
994 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Glob);
995 movq Ptr_Glob(%rip), %rax # Ptr_Glob, Ptr_Glob
996 # dry.c:753: REG Rec_Pointer Next_Record = Ptr_Val_Par->Ptr_Comp;
997 movq (%rdi), %rbx # Ptr_Val_Par_14(D)->Ptr_Comp, Next_Record
998 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Glob);
999 movdqu (%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
1000 movups %xmm0, (%rbx) # *Ptr_Glob.95_1, *Next_Record_15
1001 movdqu 16(%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
1002 movups %xmm0, 16(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
1003 movdqu 32(%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
1004 movups %xmm0, 32(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
1005 movq 48(%rax), %rax # *Ptr_Glob.95_1, *Ptr_Glob.95_1
1006 movq %rax, 48(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
1007 # dry.c:759: Ptr_Val_Par->variant.var_1.Int_Comp = 5;
1008 movl $5, 16(%rdi) #, Ptr_Val_Par_14(D)->variant.var_1.Int_Comp
1009 # dry.c:761: = Ptr_Val_Par->variant.var_1.Int_Comp;
1010 movl $5, 16(%rbx) #, Next_Record_15->variant.var_1.Int_Comp
```

03

```
PartA > X86_64_GCC_and_Clang > asm-snippets_x86_64_GCC > asm dry_03.s
10 Proc_1:
11 .LFB23:
12 .cfi_startproc
13 pushq %rbp #
14 .cfi_def_cfa_offset 16
15 .cfi_offset 6, -16
16 movq %rdi, %rbp # Ptr_Val_Par, Ptr_Val_Par
17 pushq %rbx #
18 .cfi_def_cfa_offset 24
19 .cfi_offset 3, -24
20 subq $8, %rsp #,
21 .cfi_def_cfa_offset 32
22 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Glob);
23 movq Ptr_Glob(%rip), %rax # Ptr_Glob, Ptr_Glob
24 # dry.c:753: REG Rec_Pointer Next_Record = Ptr_Val_Par->Ptr_Comp;
25 movq (%rdi), %rbx # Ptr_Val_Par_14(D)->Ptr_Comp, Next_Record
26 # dry.c:758: structassign (*Ptr_Val_Par->Ptr_Comp, *Ptr_Glob);
27 movdqu (%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
28 movups %xmm0, (%rbx) # *Ptr_Glob.95_1, *Next_Record_15
29 movdqu 16(%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
30 movups %xmm0, 16(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
31 movdqu 32(%rax), %xmm0 # *Ptr_Glob.95_1, *Ptr_Glob.95_1
32 movups %xmm0, 32(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
33 movq 48(%rax), %rax # *Ptr_Glob.95_1, *Ptr_Glob.95_1
34 movq %rax, 48(%rbx) # *Ptr_Glob.95_1, *Next_Record_15
35 # dry.c:759: Ptr_Val_Par->variant.var_1.Int_Comp = 5;
36 movl $5, 16(%rdi) #, Ptr_Val_Par_14(D)->variant.var_1.Int_Comp
37 # dry.c:761: = Ptr_Val_Par->variant.var_1.Int_Comp;
38 movl $5, 16(%rbx) #, Next_Record_15->variant.var_1.Int_Comp
39 # dry.c:762: Next_Record->Ptr_Comp = Ptr_Val_Par->Ptr_Comp;
40 movq (%rdi), %rax # Ptr_Val_Par_14(D)->Ptr_Comp, _4
```

I have split this comparison into 2 parts (**O0 vs O1**) and (**O2 vs O3**), In both cases I have taken the Proc_1 Function for my comparison.

Proc_1 in O0 vs O1

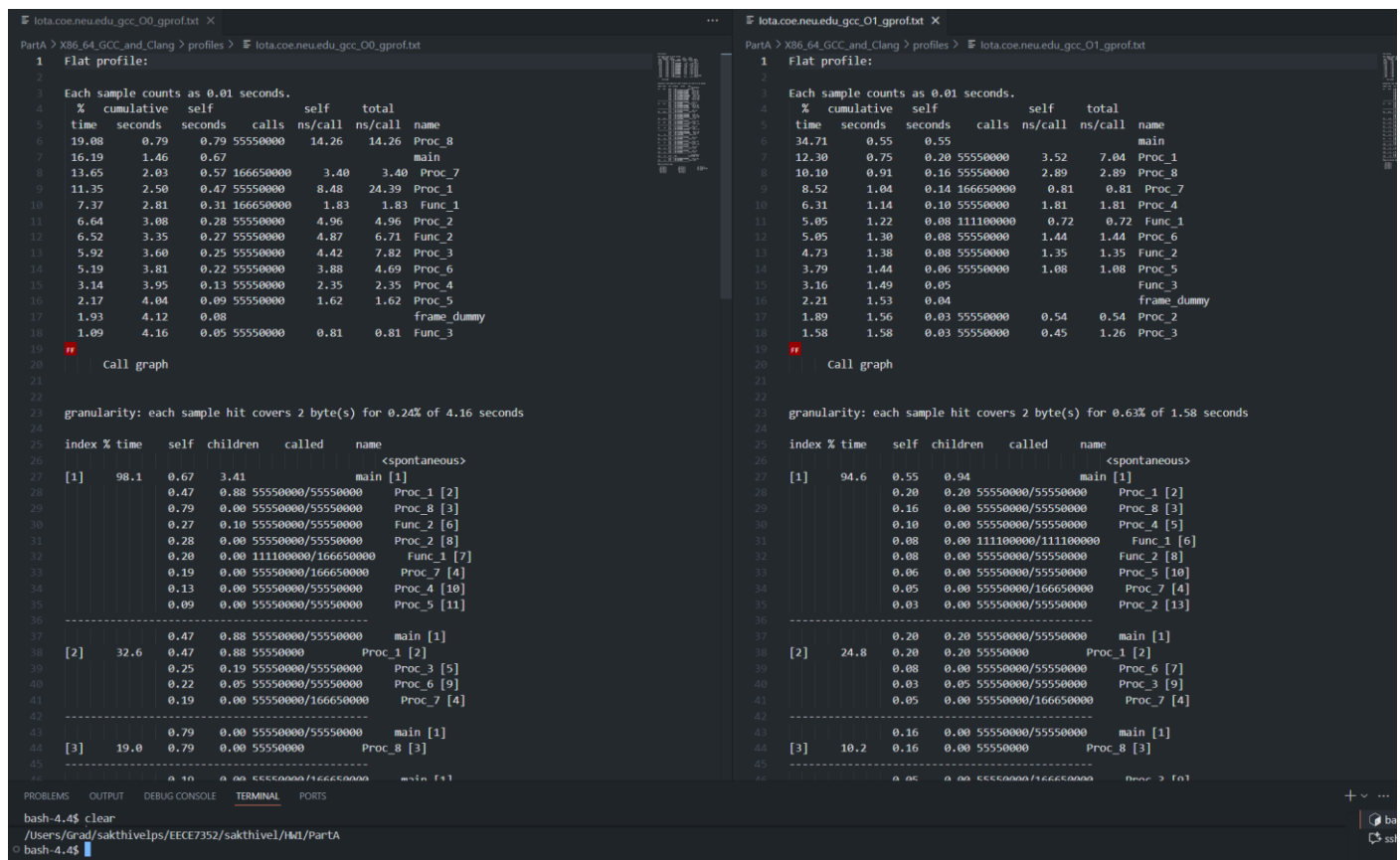
- In O0, every parameter is spilled to the stack
- Funct calls like **proc_3** are **left as calls**, where inlining could have been done.
- In O1 **locals are kept in registers** instead of spilling to memory
- Struct copies use **movdqu** or **movups** instead of word-by-word moves thus reducing memory traffic significantly

Proc_1 in O2 vs O3

- Aggressive register allocation is done in O2 compared to O1.
- Conditional checks are compiled into **Test** functions
- O3 is very similar to O2 , with some minor differences in instruction reordering and **preloading constants with 'moveaps'**
- Overall there isn't much difference between O2 vs O3 compared to O0 vs O1

X86_64 GPROF Analysis

O0 and O1



O2 and O3

lota.coe.neu.edu_gcc_O2_gprof.txt

PartA > X86_64.GCC and Clang > profiles > Iota.coe.neu.edu_gcc_O2_gprof.txt

1 Flat profile:

2

3 Each sample counts as 0.01 seconds.

4 % cumulative self self total

5 time seconds seconds calls ns/call ns/call name

6 28.84 0.20 0.20 Proc_1

7 21.63 0.35 0.15 Proc_8

8 18.74 0.48 0.13 55550000 2.36 2.36 Proc_6

9 14.42 0.59 0.10 55550000 1.82 1.82 Proc_7

10 8.65 0.65 0.06 Func_2

11 5.77 0.69 0.04 Func_1

12 1.44 0.70 0.01 Func_3

13 1.44 0.71 0.01 frame_dummy

14

15 Call graph

16

17

18 granularity: each sample hit covers 2 byte(s) for 1.42% of 0.71 seconds

19

20 index % time self children called name

21 <spontaneous>

22 [1] 61.4 0.20 0.23 Proc_1 [1]

23 0.13 0.00 55550000/55550000 Proc_6 [3]

24 0.10 0.00 55550000/55550000 Proc_7 [4]

25 -----

26 <spontaneous>

27 [2] 21.4 0.15 0.00 Proc_8 [2]

28 -----

29 0.13 0.00 55550000/55550000 Proc_1 [1]

30 0.13 0.00 55550000 Proc_6 [3]

31 -----

32 0.10 0.00 55550000/55550000 Proc_1 [1]

33 [4] 14.3 0.10 0.00 55550000 Proc_7 [4]

34 -----

35 <spontaneous>

36 [5] 8.6 0.06 0.00 Func_2 [5]

37 -----

38 <spontaneous>

39 [6] 5.7 0.04 0.00 Func_1 [6]

40 -----

41 <spontaneous>

42 [7] 1.4 0.01 0.00 Func_3 [7]

43 -----

44 <spontaneous>

45 [8] 1.4 0.01 0.00 frame_dummy [8]

46

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

bash-4.4\$ clear

/Users/Grad/sakthivelps/EECE7352/sakthivel/HW1/PartA

bash-4.4\$

lota.coe.neu.edu_gcc_O3_gprof.txt

PartA > X86_64.GCC and Clang > profiles > Iota.coe.neu.edu_gcc_O3_gprof.txt

1 Flat profile:

2

3 Each sample counts as 0.01 seconds.

4 % cumulative self self total

5 time seconds seconds calls ns/call ns/call name

6 27.37 0.16 0.16 Proc_1

7 18.82 0.27 0.11 55550000 2.00 2.00 Proc_6

8 16.25 0.37 0.10 Proc_8

9 11.98 0.44 0.07 Func_2

10 11.98 0.51 0.07 frame_dummy

11 7.70 0.56 0.05 Func_1

12 5.13 0.59 0.03 55550000 0.55 0.55 Proc_7

13 1.71 0.60 0.01 Func_3

14

15 Call graph

16

17

18 granularity: each sample hit covers 2 byte(s) for 1.68% of 0.60 seconds

19

20 index % time self children called name

21 <spontaneous>

22 [1] 50.8 0.16 0.14 Proc_1 [1]

23 0.11 0.00 55550000/55550000 Proc_6 [2]

24 0.03 0.00 55550000/55550000 Proc_7 [7]

25 -----

26 0.11 0.00 55550000/55550000 Proc_1 [1]

27 [2] 18.6 0.11 0.00 55550000 Proc_6 [2]

28 -----

29 <spontaneous>

30 [3] 16.1 0.10 0.00 Proc_8 [3]

31 -----

32 <spontaneous>

33 [4] 11.9 0.07 0.00 Func_2 [4]

34 -----

35 <spontaneous>

36 [5] 11.9 0.07 0.00 frame_dummy [5]

37 -----

38 <spontaneous>

39 [6] 7.6 0.05 0.00 Func_1 [6]

40 -----

41 0.03 0.00 55550000/55550000 Proc_1 [1]

42 [7] 5.1 0.03 0.00 55550000 Proc_7 [7]

43 -----

44 <spontaneous>

45 [8] 1.7 0.01 0.00 Func_3 [8]

46

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

bash-4.4\$ clear

/Users/Grad/sakthivelps/EECE7352/sakthivel/HW1/PartA

bash-4.4\$

GPROF results for X 86 (GCC and Clang)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_gcc_O0_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

19.08 0.79 0.79 55550000 14.26 14.26 Proc_8

16.19 1.46 0.67 main

13.65 2.03 0.57 166650000 3.40 3.40 Proc_7

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_gcc_O1_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

34.71 0.55 0.55 main

12.30 0.75 0.20 55550000 3.52 7.04 Proc_1

10.10 0.91 0.16 55550000 2.89 2.89 Proc_8

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_gcc_O2_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

28.84 0.20 0.20 Proc_1

21.63 0.35 0.15 Proc_8

18.74 0.48 0.13 55550000 2.36 2.36 Proc_6

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_gcc_O3_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

27.37 0.16 0.16 Proc_1

18.82 0.27 0.11 55550000 2.00 2.00 Proc_6

16.25 0.37 0.10 Proc_8

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_clang_O0_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

20.16 0.76 0.76 main

16.97 1.40 0.64 55550000 11.55 11.55 Proc_8

14.59 1.95 0.55 166650000 3.31 3.31 Func_1

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_clang_O1_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

19.98 0.49 0.49 main

15.29 0.87 0.38 166650000 2.26 2.26 Proc_7

12.64 1.18 0.31 55550000 5.60 17.25 Proc_1

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_clang_O2_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

37.62 0.66 0.66 main

20.52 1.02 0.36 166650000 2.17 2.17 Proc_7

11.68 1.23 0.21 55550000 3.70 3.70 Proc_8

● bash-4.4\$ grep -A100 "Flat profile" Iota.coe.neu.edu_clang_O3_gprof.txt | awk 'NR>5 && \$1 ~ /^[0-9]/ {print \$0}' | head -n3

34.48 0.50 0.50 main

17.07 0.74 0.25 166650000 1.47 1.47 Proc_7

13.93 0.94 0.20 55550000 3.61 3.61 Proc_8

● bash-4.4\$

Below are the three most frequently executed functions along with their percentage of the entire execution time for X86_64

For **O0** – proc_8 (19.08), main (16.19), proc_7 (13.65)
 For **O1** – main (34.71), proc_1 (12.30), proc_8 (10.10)
 For **O2** – proc_1(28.84), proc_8 (21.63), proc_6 (18.74)
 For **O3** – proc_1 (27.37), proc_ 6 (18.82), proc_8 (16.25)

GPROF results for ARM (GCC)

```
sakthivelps@degrees:~$ ls
Desktop  Documents  Downloads  EECE7352  Music  Pictures  Public  Templates  tsclient  Videos  Win10Files  www
sakthivelps@degrees:~$ cd EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles/
sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles$ grep -A100 "Flat profile" degrees_gcc_v8_00_gprof.txt | awk 'NR>5 && $1 ~ /^[0-9]/ {print $0}' | head -n3
21.92    0.32    0.32  5550000    57.66    57.66    Proc_8
19.18    0.60    0.28                                     main
15.07    0.82    0.22  5550000    39.64    76.88    Proc_1
sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles$ grep -A100 "Flat profile" degrees_gcc_v8_01_gprof.txt | awk 'NR>5 && $1 ~ /^[0-9]/ {print $0}' | head -n3
29.47    0.10    0.10                                     main
14.73    0.15    0.05  16650000    3.01     3.01    Proc_7
14.73    0.20    0.05  5550000     9.03    18.65    Proc_1
sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles$ grep -A100 "Flat profile" degrees_gcc_v8_02_gprof.txt | awk 'NR>5 && $1 ~ /^[0-9]/ {print $0}' | head -n3
23.23    0.40    0.40                                     Proc_8
17.13    0.70    0.30  5550000     5.34     5.34    Proc_3
17.13    0.99    0.30                                     Proc_1
sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles$ grep -A100 "Flat profile" degrees_gcc_v8_03_gprof.txt | awk 'NR>5 && $1 ~ /^[0-9]/ {print $0}' | head -n3
29.45    0.51    0.51                                     Proc_1
25.98    0.96    0.45  5550000     8.14     8.14    Proc_7
18.48    1.29    0.32  5550000     5.79     5.79    Proc_6
sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartA/ARM_V8_GCC/profiles$
```

Similarly for ARM

For **O0** – proc_8 (21.92), main (15.07), proc_6 (13.47)
 For **O1** – main (29.47), proc_7 (14.73), proc_1 (13.65)
 For **O2** – proc_3 (20.80), proc_1 (17.13), proc_7 (17.13)
 For **O3** – proc_1 (29.45), proc_7 (25.98), proc_6 (18.48)

from the above data we can see that as we optimize the code the % of execution time also changes.

x86_64 (GCC/Clang results)

- O0: time is spread across proc_8, main, and other helpers. Function call overhead is visible.
- O1: main absorbs more time (34%), while proc_7 and other helpers still take a share, meaning the compiler has not fully collapsed functions.
- O2/O3: percentages collapse strongly into proc_1 (27–28%) with proc_6 and proc_8 next. With Clang, the effect is sharper and proc_1 alone dominates more than 50% at O3.

Overall, on x86, **optimizations consolidate execution into fewer hotspots compared to ARM**, reflecting more aggressive inlining and vectorization.

ARM_v8 (GCC results)

- O0: proc_8, main, and proc_7 dominate. Percentages are spread because no inlining is applied.
- O1: main becomes the largest contributor (35%), while smaller procedures like proc_7 shrink. The compiler starts inlining helper functions.
- O2/O3: execution consolidates into proc_1 (27–29%) and other compute-heavy kernels such as proc_6 and proc_8. Overhead functions are minimized.

Overall, on ARM, as optimization increases, **percentages shift from many scattered functions toward a few compute-heavy kernels**, but the distribution still involves multiple functions.

Overall Summary for Part A:

- From the above comparisons of running Dhrystone on various ISAs under different optimization levels, we can see a general trend of doubling the performance levels (Dhrystone/sec) when we move from no optimization (O0) to O1, and a significant increase from O2 to O3.
- This experiment has shown us that **higher levels of optimization don't necessarily translate to improved levels of performance**, and this was evident in O2 vs O3.
- In Clang vs GCC, we were able to witness, **clang having a slight upper hand**, shows compiler could also play a major role in determining the performance.
- In profiling, we were able to see **ARM increased the optimization by distributing the functions**, whereas X86 followed a close knitted approach.